

Investigación

Control de Versiones

- **Definición:** Sistema que registra los cambios realizados en un archivo o conjunto de archivos a lo largo del tiempo, permitiendo recuperar versiones específicas en el futuro.
- **Utilidad:** Permite revertir archivos a un estado anterior, comparar cambios, ver quién modificó algo que causó un error y trabajar en equipo sin sobrescribir el código de otros.
- **Ejemplo práctico:** Un equipo de desarrollo trabaja en una aplicación de comercio electrónico. Si una nueva actualización rompe el sistema de pagos, el control de versiones permite regresar la aplicación al estado estable que tenía el día anterior en cuestión de segundos.
- **Fuente:** <https://www.atlassian.com/es/git/tutorials/what-is-version-control>

Git

- **Definición:** Un sistema de control de versiones **distribuido**, gratuito y de código abierto, diseñado para gestionar proyectos desde pequeños hasta muy grandes con rapidez y eficiencia.
- **Utilidad:** Al ser distribuido, cada desarrollador tiene una copia local completa del historial del proyecto en su máquina, lo que permite trabajar sin conexión a internet y realizar operaciones de forma local y veloz.
- **Ejemplo práctico:** Instalas Git en tu computadora, abres la terminal dentro de la carpeta de tu proyecto y ejecutas `git init` para empezar a rastrear los cambios de tus archivos de código.
- **Fuente:** <https://www.atlassian.com/es/git/tutorials/what-is-git>

GitHub

- **Definición:** Plataforma de alojamiento en la nube que utiliza Git para el control de versiones y añade herramientas de colaboración, como gestión de tareas, seguimiento de errores y revisión de código.
- **Utilidad:** Sirve como el nodo centralizado donde los desarrolladores suben sus proyectos para respaldar la información, colaborar con otros programadores en todo el mundo y automatizar flujos de trabajo.
- **Ejemplo práctico:** Creas una cuenta en github.com para subir el código de tu proyecto escolar, permitiendo que tu compañero de equipo lo descargue en su casa para que ambos avancen simultáneamente.
- **Fuente:**
<https://docs.github.com/es/get-started/start-your-journey/about-github-and-git>

Repositorio Local

- **Definición:** El directorio de almacenamiento del proyecto que se encuentra físicamente dentro de la computadora del desarrollador, incluyendo todo el historial de cambios de Git (la carpeta oculta `.git`).
- **Utilidad:** Permite al desarrollador hacer pruebas, crear nuevas funciones y guardar su progreso de forma privada y sin necesidad de tener acceso a internet.
- **Ejemplo práctico:** Modificas el archivo `index.html` en tu editor de código (como VS Code) dentro de tu laptop; esos cambios viven únicamente en tu repositorio local hasta que decidas enviarlos a la nube.
- **Fuente:** <https://aws.amazon.com/es/what-is/repo/>

Repositorio Remoto

- **Definición:** Una versión del proyecto que se encuentra alojada en un servidor de internet o en una red compartida (como GitHub, GitLab o Bitbucket).
- **Utilidad:** Funciona como la "fuente de la verdad" del proyecto, permitiendo que el equipo comparta código de manera segura y sirviendo como respaldo en caso de que falle la computadora local de algún desarrollador.
- **Ejemplo práctico:** Tras enlazar tu proyecto local con una URL de GitHub, usas el comando `git push origin main` para enviar todos tus avances al repositorio en la nube.
- **Fuente:** <https://tms-opensource.com/blog/posts/what-is-a-remote-repository/>

Commit

- **Definición:** Una instantánea o "foto" del estado de los archivos del proyecto en un momento específico, guardada de forma permanente en el historial de Git con un mensaje descriptivo.
- **Utilidad:** Funciona como un punto de guardado en un videojuego. Si arruinas el código más adelante, siempre puedes volver exactamente al estado de este commit.
- **Ejemplo práctico:** Terminas de programar el formulario de inicio de sesión, guardas los cambios y ejecutas en la terminal:
- Bash

`git commit -m "feat: agregar validación al formulario de login"`

- **Fuente:** <https://www.atlassian.com/es/git/tutorials/saving-changes/git-commit>

Branch (Rama)

- **Definición:** Una línea de desarrollo independiente que se desvía de la línea principal (generalmente llamada `main` o `master`), permitiendo realizar modificaciones sin alterar el código estable.
- **Utilidad:** Permite desarrollar nuevas funcionalidades, experimentar o reparar errores en paralelo sin interrumpir el trabajo de los demás miembros del equipo.
- **Ejemplo práctico:** Creas una rama llamada `desarrollo-carrito` usando `git checkout -b desarrollo-carrito` para programar el carrito de compras

sin arriesgarte a dañar la página web principal que ya está funcionando en producción.

- **Fuente:** <https://www.atlassian.com/es/git/tutorials/saving-changes/git-commit>

Merge

- **Definición:** La operación que fusiona el historial y los cambios de una rama secundaria dentro de otra rama (comúnmente la rama principal).
- **Utilidad:** Permite incorporar de manera oficial las nuevas características o correcciones ya terminadas y probadas al flujo de trabajo principal del proyecto.
- **Ejemplo práctico:** Una vez que terminaste el carrito de compras en la rama `desarrollo-carrito`, regresas a la rama principal (`git checkout main`) y ejecutas `git merge desarrollo-carrito` para integrar el código nuevo.
- **Fuente:** Chacon, S., & Straub, B. (2014). *Pro Git*. Apress.

Pull Request (PR)

- **Definición:** Una petición formal en plataformas como GitHub donde un desarrollador solicita que sus cambios de una rama sean fusionados en la rama principal de un repositorio.
- **Utilidad:** Facilita la revisión de código (*code review*). Los demás miembros del equipo pueden ver qué cambió, comentar líneas específicas, proponer mejoras y aprobar el código antes de que afecte al proyecto.
- **Ejemplo práctico:** Subes tu rama a GitHub y das clic en "New Pull Request". Tu líder técnico revisa el código en la plataforma, te sugiere cambiar una variable por seguridad, lo corriges y, tras su aprobación, presiona "Merge".
- **Fuente:** Bell, P., & Beer, B. (2014). *Introducing GitHub: A Non-Technical Guide*. O'Reilly Media.

Fork

- **Definición:** Una copia exacta de un repositorio ajeno que se guarda directamente dentro de tu propia cuenta de GitHub.

- **Utilidad:** Te permite experimentar libremente con el proyecto de otra persona o proponer mejoras a proyectos de código abierto (*open-source*) sin modificar directamente el repositorio original.
- **Ejemplo práctico:** Encuentras una librería de Angular en GitHub que tiene un error. Haces un *fork* a tu cuenta, corriges el error en tu copia y luego envías un Pull Request al dueño original para que incorpore tu solución.
- **Fuente:** Bell, P., & Beer, B. (2014). *Introducing GitHub: A Non-Technical Guide*. O'Reilly Media.

Conflictos de Integración

- **Definición:** Situación que ocurre cuando Git no puede fusionar automáticamente dos ramas porque se han realizado modificaciones en la misma línea del mismo archivo en ambas ramas, o porque un archivo fue eliminado en una y editado en otra.
- **Utilidad:** Git detiene el proceso para evitar que se pierda información importante por accidente, obligando al desarrollador a decidir manualmente qué cambios conservar.
- **Ejemplo práctico:** Tu compañero modificó la línea 15 de `styles.css` para poner el fondo azul, y tú modificaste la misma línea 15 para ponerlo verde. Al hacer el *merge*, Git se detiene y te muestra ambas opciones en el editor de texto para que elijas cuál debe quedarse.
- **Fuente:** Chacon, S., & Straub, B. (2014). *Pro Git*. Apress.

Integración Continua (CI)

- **Definición:** Práctica de desarrollo de software donde los miembros del equipo integran su código frecuentemente (al menos una vez al día) en un repositorio compartido. Cada integración es verificada de forma automática mediante la compilación y pruebas unitarias.
- **Utilidad:** Detecta errores de integración lo antes posible, evitando el temido "infierno de la integración" al final del proyecto y asegurando que el código siempre compile bien.

- **Ejemplo práctico:** Cada vez que haces un `git push` a GitHub, un servidor externo toma tu código automáticamente, instala las dependencias y ejecuta las pruebas de la aplicación para confirmar que tu cambio no rompió nada existente.
- **Fuente:** Humble, J., & Farley, D. (2010). *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley Professional.

Entrega Continua (Continuous Delivery)

- **Definición:** Extensión de la Integración Continua. Consiste en automatizar todo el proceso de preparación del software para su lanzamiento, asegurando que el repositorio de código esté siempre en un estado óptimo para ser desplegado en producción.
- **Utilidad:** Reduce el riesgo y el esfuerzo de lanzar nuevas versiones, permitiendo que el despliegue al cliente final se realice en cualquier momento con tan solo presionar un botón de forma manual.
- **Ejemplo práctico:** Tras pasar las pruebas automáticas de CI, el sistema empaqueta la aplicación en un contenedor (como Docker) y la deja lista en un servidor de pruebas (Staging) esperando que el gerente de producto autorice el lanzamiento definitivo.
- **Fuente:** Humble, J., & Farley, D. (2010). *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley Professional.

Despliegue Continuo (Continuous Deployment)

- **Definición:** Práctica donde cada cambio que pasa con éxito todas las etapas del flujo de automatización y pruebas es lanzado automáticamente al entorno de producción (clientes reales) sin intervención humana manual.
- **Utilidad:** Maximiza la velocidad de entrega de valor al usuario. Un cambio o corrección puede pasar de la computadora del desarrollador a estar disponible globalmente en minutos.

- **Ejemplo práctico:** Modificas un texto de la interfaz de usuario en tu código, haces commit y push; el sistema corre las pruebas automáticas, las aprueba y, sin que nadie intervenga, actualiza directamente la página web en vivo que usan tus clientes.
- **Fuente:** Kim, G., Humble, J., Debois, P., & Willis, J. (2016). *The DevOps Handbook*. IT Revolution.

GitHub Actions

- **Definición:** La herramienta nativa de GitHub para la automatización de flujos de trabajo (*workflows*) directamente dentro del ciclo de vida del desarrollo de software.
- **Utilidad:** Permite implementar prácticas de CI/CD escribiendo archivos de configuración sencillos en formato YAML, facilitando compilar, probar y desplegar el código desde el mismo lugar donde está alojado el repositorio.
- **Ejemplo práctico:** Creas un archivo en la ruta `.github/workflows/main.yml` que le dice a GitHub: "Cada vez que haya un Pull Request a la rama main, ejecuta `npm run test` y `npm run build`".
- **Fuente:** GitHub Docs. (2026). *Understanding GitHub Actions*. GitHub Inc.

Demostración Práctica

1: Creación del repositorio en GitHub

- **Actividad realizada:** Se procedió a la creación y configuración inicial de un nuevo repositorio público/privado en la plataforma GitHub. Se definió el nombre del proyecto, se añadió una descripción inicial y se inicializó con un archivo `README.md` y un archivo `.gitignore` adecuado para el entorno de desarrollo.
- **Resultado obtenido:** Un repositorio remoto vacío y centralizado en la nube listo para recibir el código del proyecto y permitir la colaboración del equipo.

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk (*).

- ### General

Owner *  yaviracjorge / **Repository name ***

Great repository names are short and memorable. How about [super-dollop](#)?

Description

0 / 350 characters
- ### Configuration

Choose visibility *

Add README ☐ Off
READMEs can be used as longer descriptions. [About READMEs](#)

Add .gitignore
.gitignore tells git which files not to track. [About ignoring files](#)

Add license
Licenses explain how others can use your code. [About licenses](#)



Start coding with Codespaces
Add a README file and start coding in a secure, configurable, and dedicated development environment.

[Create a codespace](#)



Add collaborators to this repository
Search for people using their GitHub username or email address.

[Invite collaborators](#)

Quick setup — if you've done this kind of thing before

or

Get started by [creating a new file](#) or [uploading an existing file](#). We recommend every repository include a [README](#), [LICENSE](#), and [.gitignore](#).

...or create a new repository on the command line

```
echo "# Tarea3Devops" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/yaviracjorge/Tarea3Devops.git
git push -u origin main
```

[Copy](#)

...or push an existing repository from the command line

```
git remote add origin https://github.com/yaviracjorge/Tarea3Devops.git
git branch -M main
git push -u origin main
```

[Copy](#)

2: Creación de ramas de trabajo por cada integrante

- Actividad realizada:** Con el fin de trabajar de forma aislada y no afectar la línea principal (**main**), cada integrante del equipo creó su propia rama de desarrollo local mediante el comando `git checkout -b [nombre_rama]` (por ejemplo, `feature/login-integrante1` y `feature/registro-integrante2`).

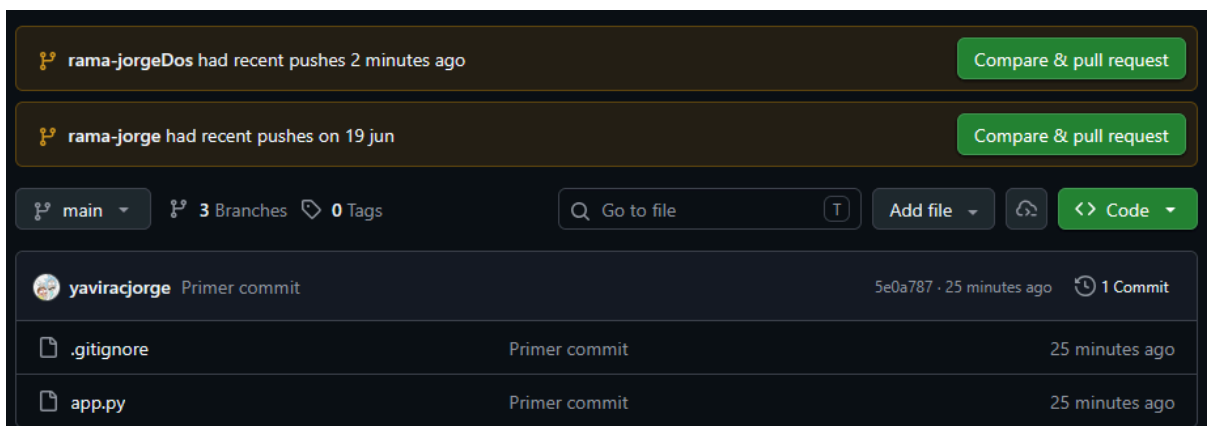
- **Resultado obtenido:** Dos líneas de tiempo independientes en Git que permiten a ambos desarrolladores avanzar simultáneamente en diferentes módulos del sistema sin interferir entre sí.

```
(env) PS C:\Users\jo\OneDrive\Escritorio\Instituto\5Semestre\Devops\tallergrupo> git checkout -b rama-jorge
Switched to a new branch 'rama-jorge'
(env) PS C:\Users\jo\OneDrive\Escritorio\Instituto\5Semestre\Devops\tallergrupo> git branch
main
* rama-jorge
(env) PS C:\Users\jo\OneDrive\Escritorio\Instituto\5Semestre\Devops\tallergrupo> |
```

```
(env) PS C:\Users\jo\OneDrive\Escritorio\Instituto\5Semestre\Devops\tallergrupo> git checkout -b rama-jorgeDos
Switched to a new branch 'rama-jorgeDos'
(env) PS C:\Users\jo\OneDrive\Escritorio\Instituto\5Semestre\Devops\tallergrupo> git branch
main
rama-jorge
* rama-jorgeDos
(env) PS C:\Users\jo\OneDrive\Escritorio\Instituto\5Semestre\Devops\tallergrupo> |
```

3: Commits realizados por ambos integrantes

- **Actividad realizada:** Tras realizar modificaciones en los archivos de código correspondientes a sus tareas, cada integrante preparó sus cambios con `git add` y realizó los puntos de guardado locales utilizando comandos `git commit -m "[mensaje descriptivo]"` siguiendo buenas prácticas de mensajería.
- **Resultado obtenido:** Registro histórico local y ordenado de los cambios de software, donde cada funcionalidad implementada queda guardada con un identificador único (Hash SHA) y el autor correspondiente.



4: Publicación de cambios al repositorio remoto

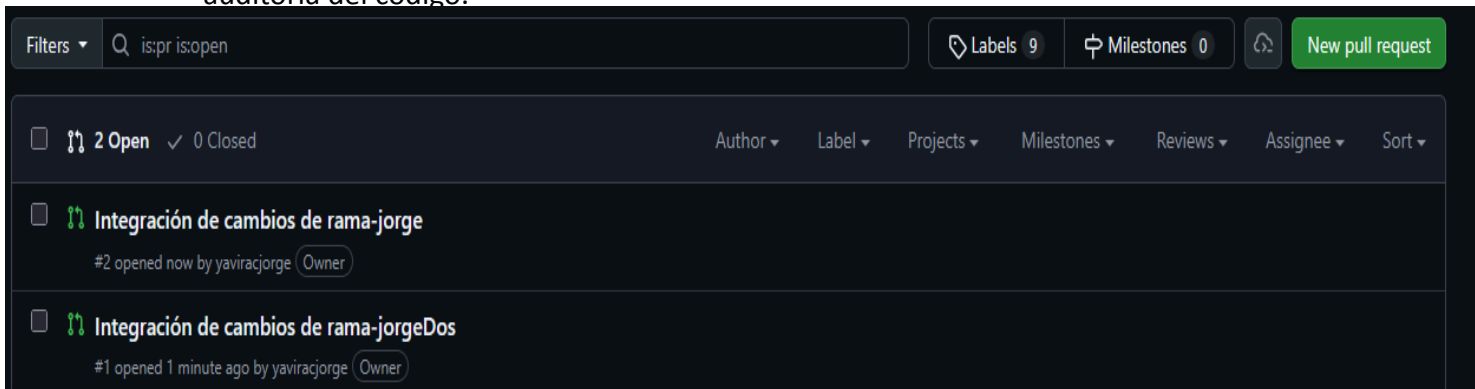
- **Actividad realizada:** Cada integrante ejecutó el comando `git push origin [nombre_de_su_rama]` para subir las ramas locales y sus respectivos commits desde sus computadoras hacia el servidor central en GitHub.
- **Resultado obtenido:** Las ramas de trabajo individuales ahora están disponibles y respaldadas en la nube (GitHub), quedando visibles para todo el equipo de desarrollo.

Default						
Branch	Updated	Check status	Behind	Ahead	Pull request	
main 	 25 minutes ago				Default	 ...
Your branches						
Branch	Updated	Check status	Behind	Ahead	Pull request	
rama-jorge 	 1 minute ago		0	1		 ...
rama-jorgeDos 	 3 minutes ago		0	1		 ...
Active branches						
Branch	Updated	Check status	Behind	Ahead	Pull request	
rama-jorge 	 1 minute ago		0	1		 ...
rama-jorgeDos 	 3 minutes ago		0	1		 ...

5: Creación de Pull Requests

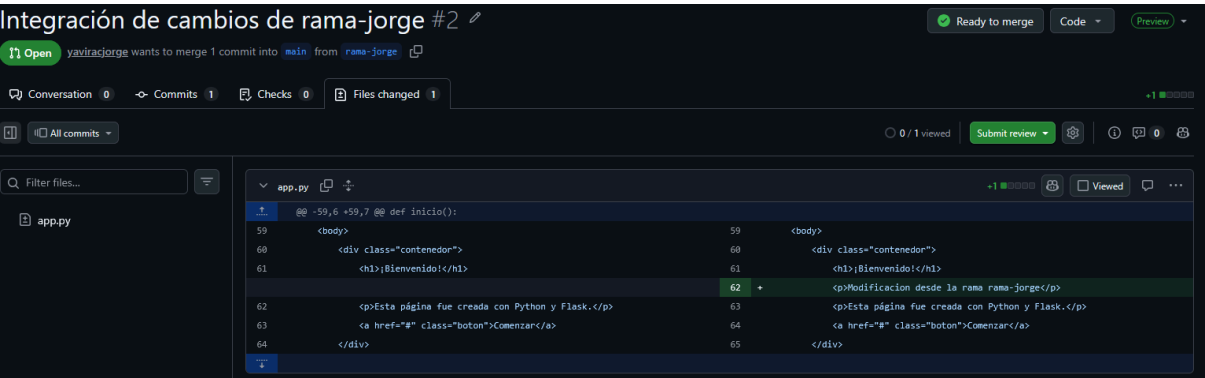
- **Actividad realizada:** Una vez finalizadas las funcionalidades en las ramas secundarias, los integrantes ingresaron a la interfaz de GitHub y abrieron una solicitud de extracción o *Pull Request* (PR) solicitando formalmente fusionar sus cambios hacia la rama principal (`main`).

- **Resultado obtenido:** Dos propuestas formales de integración documentadas en GitHub, detallando los cambios introducidos y abriendo el espacio para el debate y la auditoría del código.



6: Revisión de código (Code Review)

- **Actividad realizada:** Los integrantes del equipo actuaron como revisores mutuos. Se inspeccionaron las líneas de código añadidas y modificadas en la pestaña *Files changed* del Pull Request, dejando comentarios técnicos y sugerencias de mejora sobre el diseño del software.
- **Resultado obtenido:** Un proceso de control de calidad colaborativo donde se validó que el código cumple con los estándares del proyecto antes de ser incorporado permanentemente.



7: Aprobación de Pull Requests

- **Actividad realizada:** Tras verificar que los comentarios de la revisión fueron atendidos y comprobar que el código es correcto, el revisor asignado otorgó formalmente el estado de "Aprobado" (*Approved*) utilizando la herramienta de revisión de GitHub.
- **Resultado obtenido:** El Pull Request cambia de estado y queda formalmente autorizado para su fusión definitiva en la rama estable, cumpliendo con las políticas de gobernanza del repositorio.

Integración de cambios de rama-jorge #2

Ready to merge

Open yaviracjorge wants to merge 1 commit into main from rama-jorge

Conversation 0 Commits 1 Checks 0 Files changed 1

app.py

```

@@ -59,6 +59,7 @@ def inicio():
59     <body>
60     <div class="contenedor">
61         <h1>¡Bienvenido!</h1>
62         <p>Modificación desde la rama rama-jorge</p>
63         <p>Esta página fue creada con Python y Flask.</p>
64         <a href="#" class="boton">Comenzar</a>
65     </div>

```

Integración de cambios de rama-jorgeDos #1

Ready to merge

Open yaviracjorge wants to merge 1 commit into main from rama-jorgeDos

Conversation 0 Commits 1 Checks 0 Files changed 1

app.py

```

@@ -60,6 +60,7 @@ def inicio():
60     <div class="contenedor">
61         <h1>¡Bienvenido!</h1>
62         <p>Esta página fue creada con Python y Flask.</p>
63         <p>Modificación desde la rama rama-jorgeDos</p>
64         <a href="#" class="boton">Comenzar</a>
65     </div>
66 </body>

```

8: Ejecución del proceso de Merge

- **Actividad realizada:** Con la aprobación de los integrantes y tras comprobar que no existían impedimentos iniciales, se presionó el botón *Merge Pull Request* en GitHub para fusionar la primera de las ramas aprobadas dentro de la rama principal (**main**).
- **Resultado obtenido:** Los cambios de dicha rama secundaria se integraron exitosamente en el historial de la rama principal, cerrando automáticamente el Pull Request original.

Integración de cambios de rama-jorgeDos #1

 **Merged** yaviracjorge merged 1 commit into `main` from `rama-jorgeDos`  now

 Conversation **0**  Commits **1**  Checks **0**  Files changed **1**



yaviracjorge commented 7 minutes ago Owner ...

se agrega texto desde la rama rama-jorgeDos



 Mention @copilot in a comment to make changes to this pull request.

  Modificaicon desde al rama rama-jorgeDos 833e8d8

  **yaviracjorge** merged commit `829bc3b` into `main` now Revert



Pull request successfully merged and closed Delete branch

You're all set — the `rama-jorgeDos` branch can be safely deleted.

Integración de cambios de rama-jorge #2

 **Merged** yaviracjorge merged 1 commit into `main` from `rama-jorge`  1 minute ago

 Conversation **0**  Commits **1**  Checks **0**  Files changed **1**



yaviracjorge commented 6 minutes ago Owner ...

texto desde la rama rama-jorge



  modificacion desde la rama rama-jorge 913c8c4

  **yaviracjorge** merged commit `e2146d6` into `main` 1 minute ago Revert

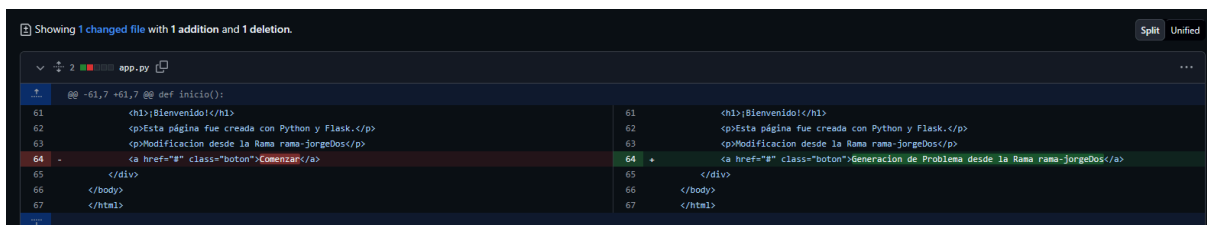


Pull request successfully merged and closed Delete branch

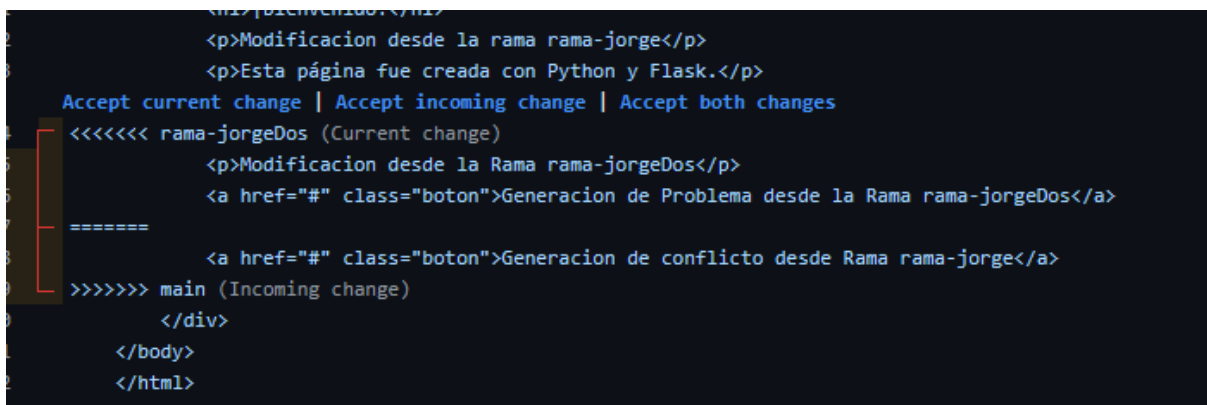
You're all set — the `rama-jorge` branch can be safely deleted.

9: Generación y resolución de conflictos de integración

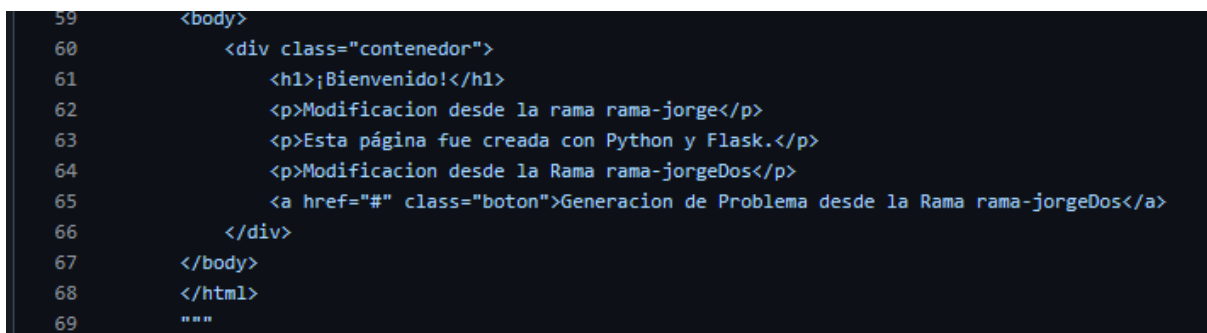
- **Actividad realizada:** Al intentar fusionar la rama del segundo integrante, Git detectó que ambos habían modificado las mismas líneas en un archivo común, deteniendo el proceso por conflicto. Se procedió a descargar los cambios localmente, abrir el archivo afectado en el editor de código, decidir manualmente qué líneas conservar, guardar, hacer un nuevo commit y subir la solución.
- **Resultado obtenido:** El conflicto fue resuelto exitosamente, permitiendo completar la fusión de la segunda rama sin pérdida de información ni sobreescrituras accidentales.



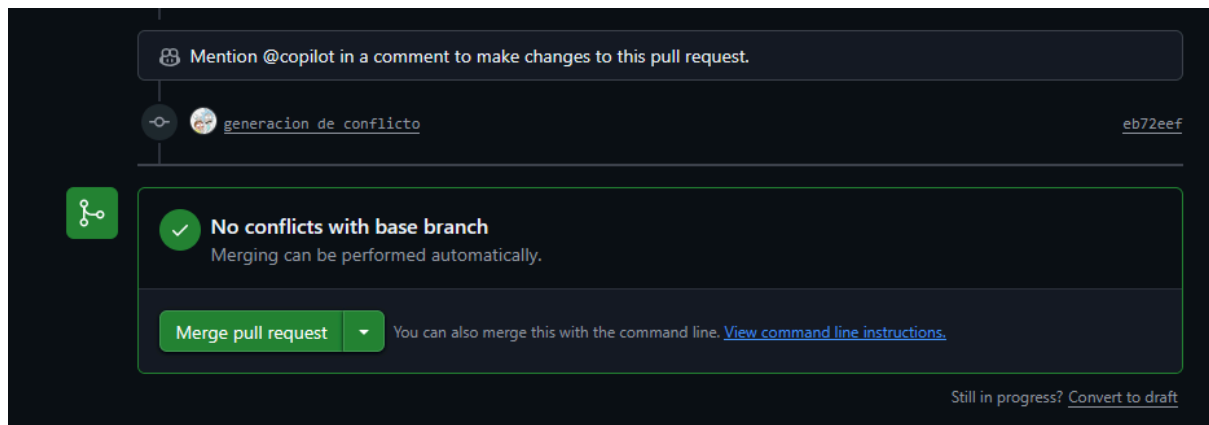
```
Showing 1 changed file with 1 addition and 1 deletion. Split Unified
app.py
61 <h1>¡Bienvenido!</h1>
62 <p>Esta página fue creada con Python y Flask.</p>
63 <p>Modificación desde la Rama rama-jorgeDos</p>
64 - <a href="#" class="boton">Comenzar</a>
65 </div>
66 </body>
67 </html>
61 <h1>¡Bienvenido!</h1>
62 <p>Esta página fue creada con Python y Flask.</p>
63 <p>Modificación desde la Rama rama-jorgeDos</p>
64 + <a href="#" class="boton">Generacion de Problema desde la Rama rama-jorgeDos</a>
65 </div>
66 </body>
67 </html>
```



```
<h1>¡Bienvenido!</h1>
<p>Modificación desde la rama rama-jorge</p>
<p>Esta página fue creada con Python y Flask.</p>
Accept current change | Accept incoming change | Accept both changes
<<<<<< rama-jorgeDos (Current change)
<p>Modificación desde la Rama rama-jorgeDos</p>
<a href="#" class="boton">Generacion de Problema desde la Rama rama-jorgeDos</a>
=====
<a href="#" class="boton">Generacion de conflicto desde Rama rama-jorge</a>
>>>>>> main (Incoming change)
</div>
</body>
</html>
```



```
59 <body>
60 <div class="contenedor">
61 <h1>¡Bienvenido!</h1>
62 <p>Modificación desde la rama rama-jorge</p>
63 <p>Esta página fue creada con Python y Flask.</p>
64 <p>Modificación desde la Rama rama-jorgeDos</p>
65 <a href="#" class="boton">Generacion de Problema desde la Rama rama-jorgeDos</a>
66 </div>
67 </body>
68 </html>
69 "" ""
```

10: Ejecución exitosa del workflow de GitHub Actions

- **Actividad realizada:** Al integrarse los cambios en **main**, se disparó de forma automatizada un flujo de trabajo (Workflow) de Integración Continua (CI) mediante GitHub Actions configurado en el archivo **.github/workflows/**. Este proceso compiló la aplicación y ejecutó las pruebas automatizadas.
- **Resultado obtenido:** Un checklist completamente en verde en la pestaña **Actions**, lo que demuestra que el código integrado compila correctamente, pasa los tests de



All workflows

Showing runs from all workflows

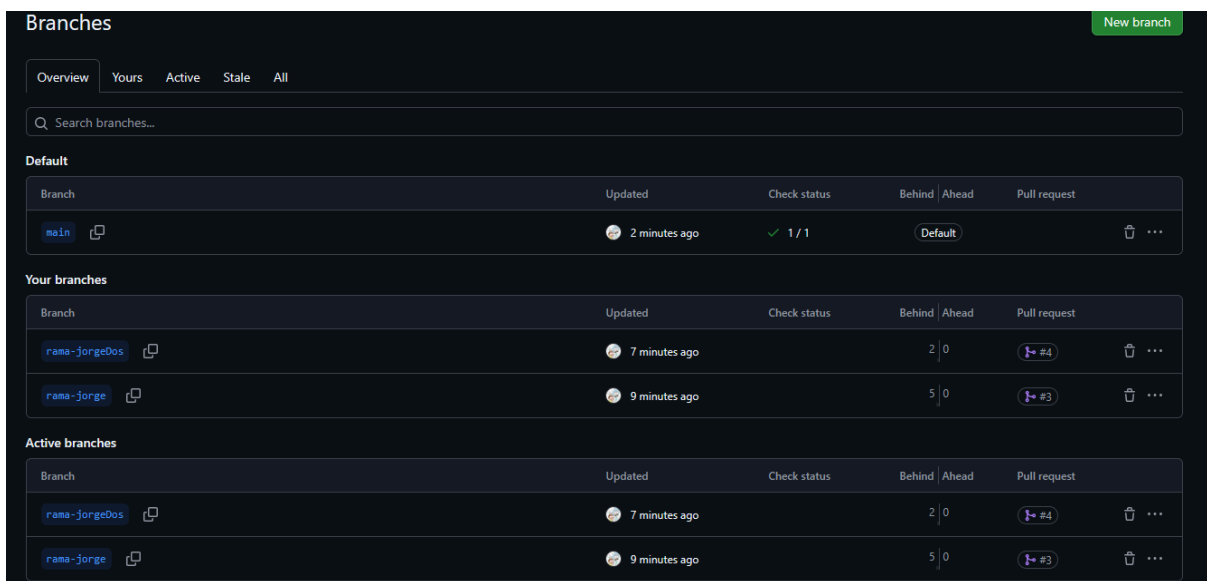
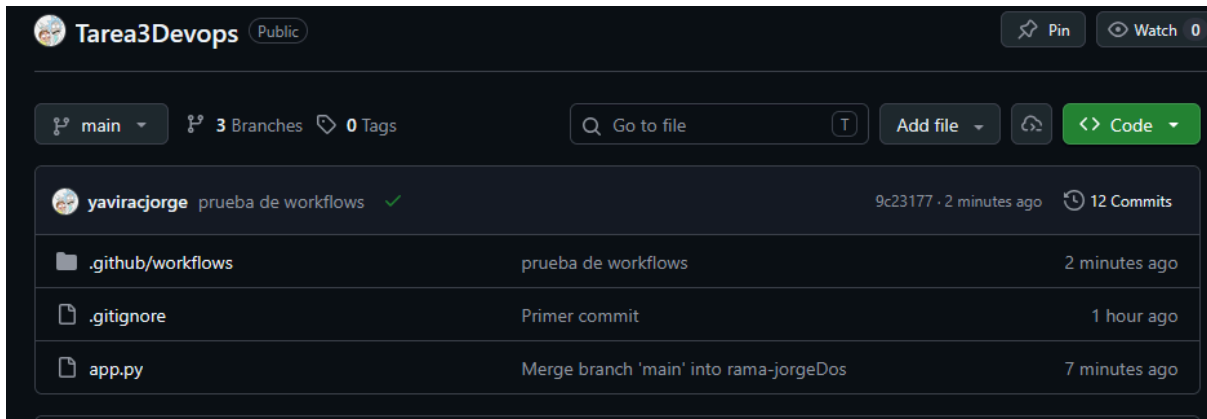
Filter workflow runs

1 workflow run

	Workflow	Event	Status	Branch	Actor
✓ prueba de workflows				main	
Python Test #1: Commit 9c23177 pushed by yaviracjorge					
			now		...
			17s		

11: Estado final del repositorio después de integrar todos los cambios

- **Actividad realizada:** Se visualizó el estado general del repositorio al concluir el flujo de trabajo. Se revisó el grafo de la red o el historial de commits en la rama **main** de GitHub una vez consolidadas todas las tareas.
- **Resultado obtenido:** Una rama principal (**main**) limpia, actualizada y robusta que contiene el código final funcional y unificado de ambos integrantes del equipo, lista para un potencial despliegue.



Nombre: Jorge Leonardo Corella Roman, Jordy Stalin Santillan Vargas

Carrera: Desarrollo de Software

Curso: 5 C

Repositorio:

<https://github.com/yaviracjorge/Tarea3Devops>

Conclusión

Esta práctica permitió comprender la importancia de Git y GitHub en el desarrollo colaborativo de software. El uso de ramas mostró cómo varios desarrolladores pueden trabajar en paralelo, mientras que los conflictos de integración demostraron ser situaciones normales que pueden resolverse sin perder información.

Además, los Pull Requests ayudaron a reforzar la revisión de código como un control de calidad antes de unir cambios. Por otro lado, la implementación de GitHub Actions evidenció el valor de la integración continua, ya que permite automatizar la verificación del código y reducir errores.

En conjunto, estas herramientas mejoran la organización, la calidad y la eficiencia en el desarrollo de software.

Conclusión 2:

En conclusión, el desarrollo de este taller colaborativo me ha permitido consolidar, tanto de forma teórica como práctica, los fundamentos esenciales del control de versiones utilizando Git y GitHub. A través del intercambio dinámico de roles con mi compañero, logré desarrollar competencias críticas como la correcta gestión de ramas, la realización de commits descriptivos y la exhaustiva revisión de código mediante Pull Requests. Además, la experiencia directa al enfrentar y resolver conflictos de integración reales, sumada a la configuración de flujos automatizados con GitHub Actions, me brindó una comprensión clara de cómo operan los flujos de trabajo en equipos de desarrollo profesional. Esta práctica no solo fortaleció mis habilidades técnicas, sino que evidenció que la organización y el trabajo en equipo son vitales para garantizar la calidad del software.

Fuentes:

<https://www.atlassian.com/es/git/tutorials/what-is-version-control>

<https://www.atlassian.com/es/git/tutorials/what-is-git>

<https://docs.github.com/es/get-started/start-your-journey/about-github-and-git>

<https://aws.amazon.com/es/what-is/repo/>

<https://tms-outsource.com/blog/posts/what-is-a-remote-repository/>

<https://www.atlassian.com/es/git/tutorials/saving-changes/git-commit> Chacon, S., & Straub, B. (2014). Pro Git. Apress. Bell, P., & Beer, B. (2014). Introducing GitHub: A Non-Technical Guide. O'Reilly Media. GitHub Docs. (2026). Understanding GitHub Actions. GitHub Inc.